Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н. Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатика, искусственный интеллект и системы управления

КАФЕДРА Системы обработки информации и управления

Лабораторная работа №1

по курсу **«Технологии разработки программного обеспечения»**

на тему

«Управление программным проектом на примере модели Agile/Scrum»

Выполнил: Давыдов В. Ю. (подпись) (дата)

Группа: ИУ5-11М

Проверил: Виноградова М. В. (подпись) (дата)

Москва, 2026 г.

1 Задание

1.1 Цель работы

Изучить возможности системы управления версиями и коллективной разработкой программного проекта; освоить методы работы с утилитой Git для управления версиями и коллективной разработкой программного проекта.

1.2 Средства выполнения

- утилита Git;
- утилита Git GUI и gitk.

1.3 Задачи

В ходе выполнения лабораторной работы были выполнены следующие пункты задания:

- запустить Git GUI и создать новый репозиторий в рабочей папке проекта;
- добавить в папку репозитория файл, зафиксировать состояние репозитория (выполнить commit);
- внести изменения в файл, зафиксировать новое состояние репозитория;
- создать новую ветку (lastname), внести в неё изменения и зафиксировать их;
- переключиться на ветку master, внести в неё изменения (исправление) и зафиксировать их;
- продемонстрировать слияние веток - разрешить возникший конфликт;
- просмотреть дерево изменений веток (историю), в том числе на примере выноса части кода в отдельные функцию и файлы;
- продемонстрировать откат изменений (revert);
- создать удалённый репозиторий на github.com и опубликовать в нём изменения.

2 Ход работы

2.1 Создание репозитория и фиксация начального состояния

С помощью контекстного меню проводника Windows в рабочей папке проекта `BMSTU_IU5_MAGA_TERM1-TRPO-Lab1` запущен Git GUI, создан новый локальный репозиторий. В каталог репозитория добавлен файл main.c с исходным кодом программы, выводящей приветствие «Hello world!». На рисунке 1 показано состояние main.c до добавления в список на фиксацию.

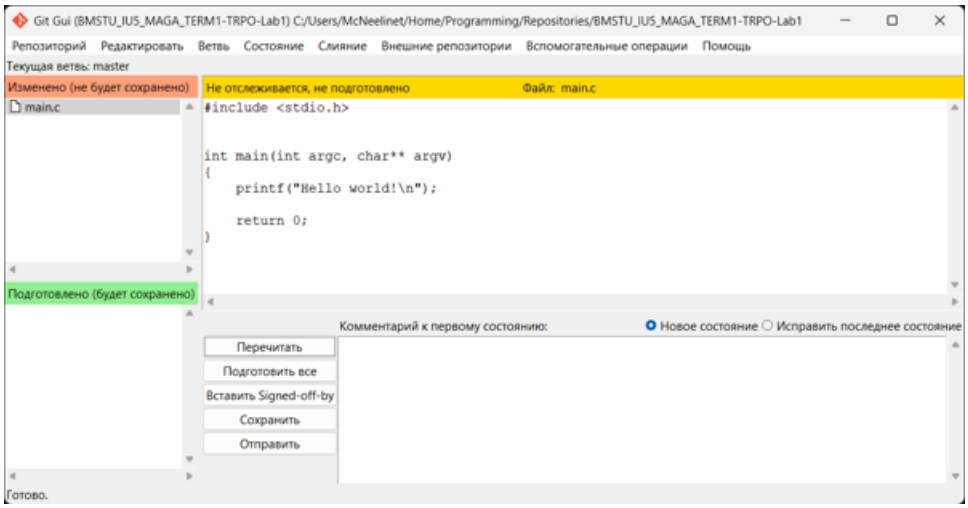


Рисунок 1 – Файл main.c перед добавлением в фиксацию (Git GUI)

После подготовки файла и ввода комментария к фиксации выполнен первый commit с сообщением «Initial commit». На рисунке 2 в окне gitk показана история репозитория и содержимое зафиксированного изменения.

2.2 Внесение изменений и повторная фиксация

Программа доработана: вместо статического текста main.c теперь запрашивает у пользователя имя (fgets) и выводит персонализированное приветствие. Изменения проиндексированы и зафиксированы отдельным коммитом «Give a personalized hello.». На рисунке 3 приведена фиксация данного изменения в gitk (показаны изменения в main.c).

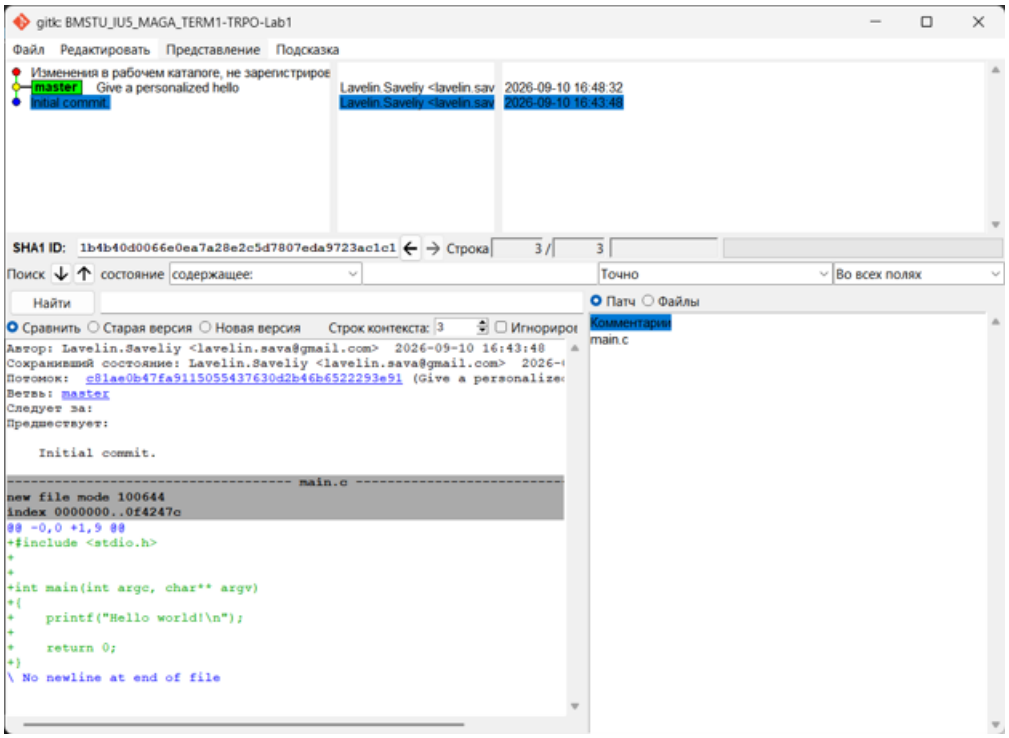


Рисунок 2 – История после фиксации начального состояния (Initial commit)

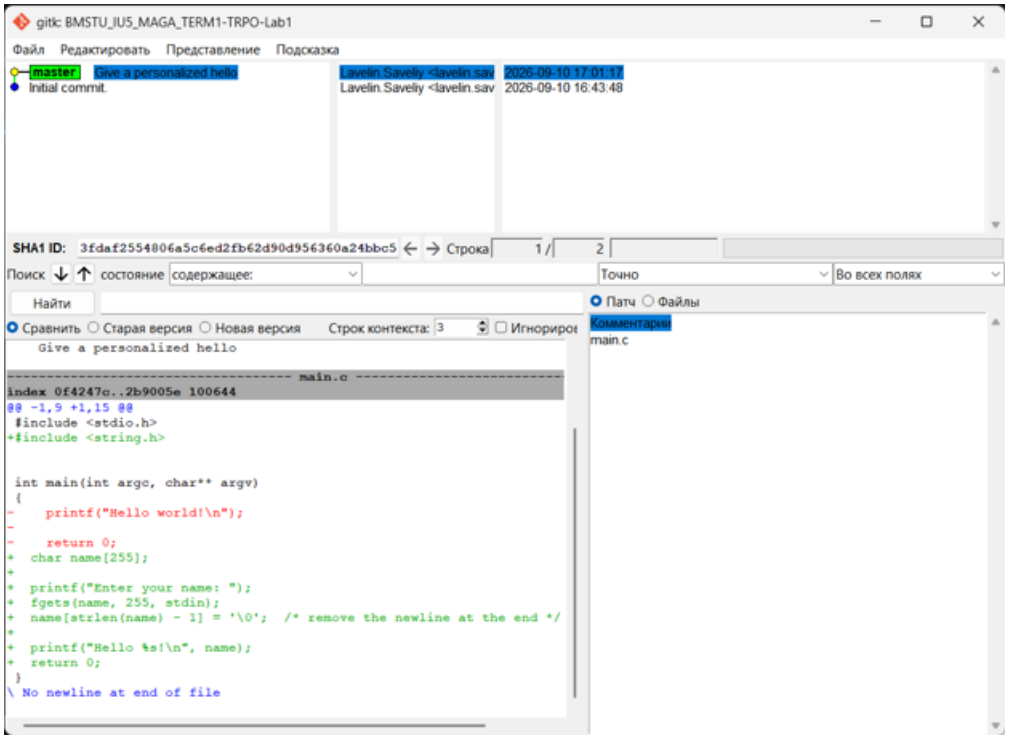


Рисунок 3 – Фиксация изменения «Give a personalized hello»

2.3 Ветвление

Для разработки нового функционала без риска для стабильной ветки создана и выбрана новая ветка «lastname». В этой ветке main.c изменён так, чтобы программа запрашивала также фамилию пользователя и выводила её вместе с именем. Изменение зафиксировано коммитом «Prompt user for last name as well.». На рисунке 4 показана фиксация изменений в ветке lastname.

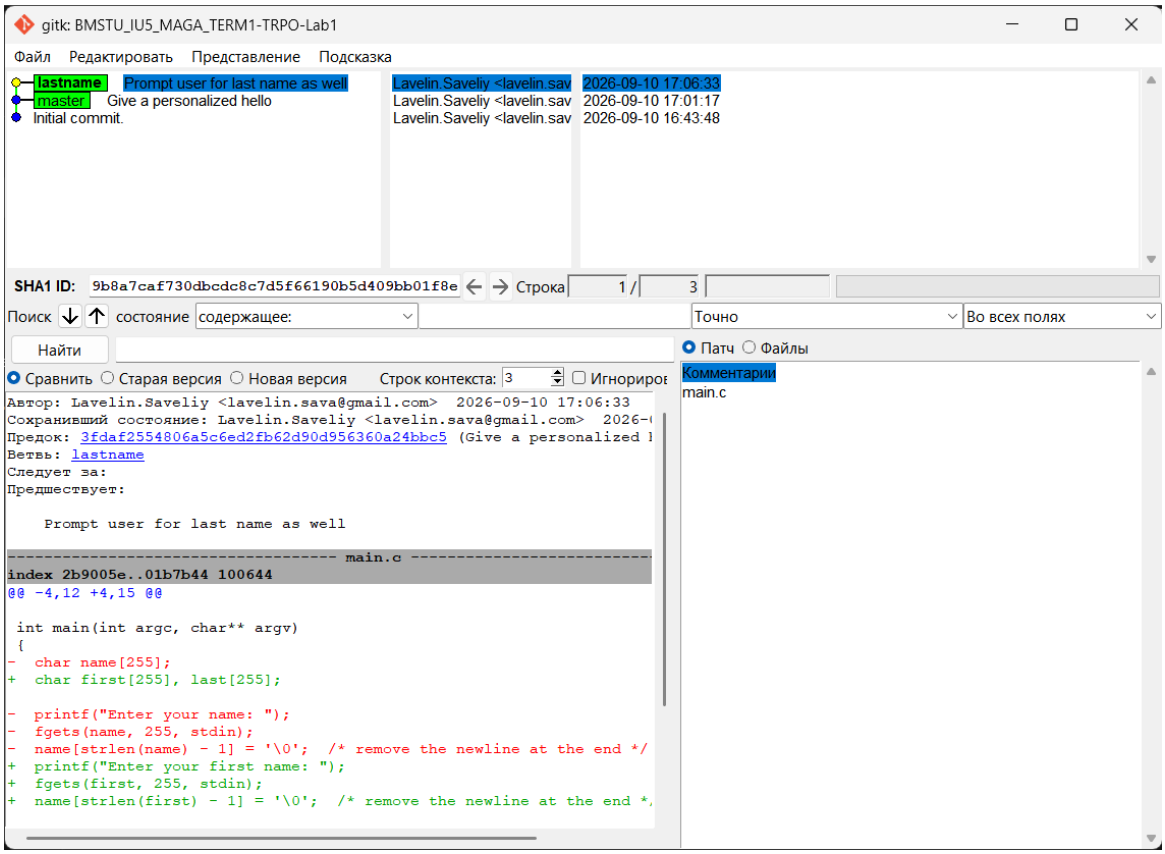


Рисунок 4 – Фиксация изменений в ветке lastname

Далее выполнено переключение обратно на ветку master. В ветке master исправлена грамматическая ошибка приветствия (добавлена запятая перед именем) и зафиксирован коммит «Use proper grammar.». На рисунке 5 приведён граф истории всех веток (Repository => Visualize All Branch History), демонстрирующий расхождение веток master и lastname от общего предка.

2.4 Слияние веток

Выполнено слияние ветки lastname в master. Так как обе ветки изменяли одну и ту же строку файла main.c, при слиянии возник конфликт. Конфликт разрешён вручную в текстовом редакторе Git GUI: в итоговом коде оставлены

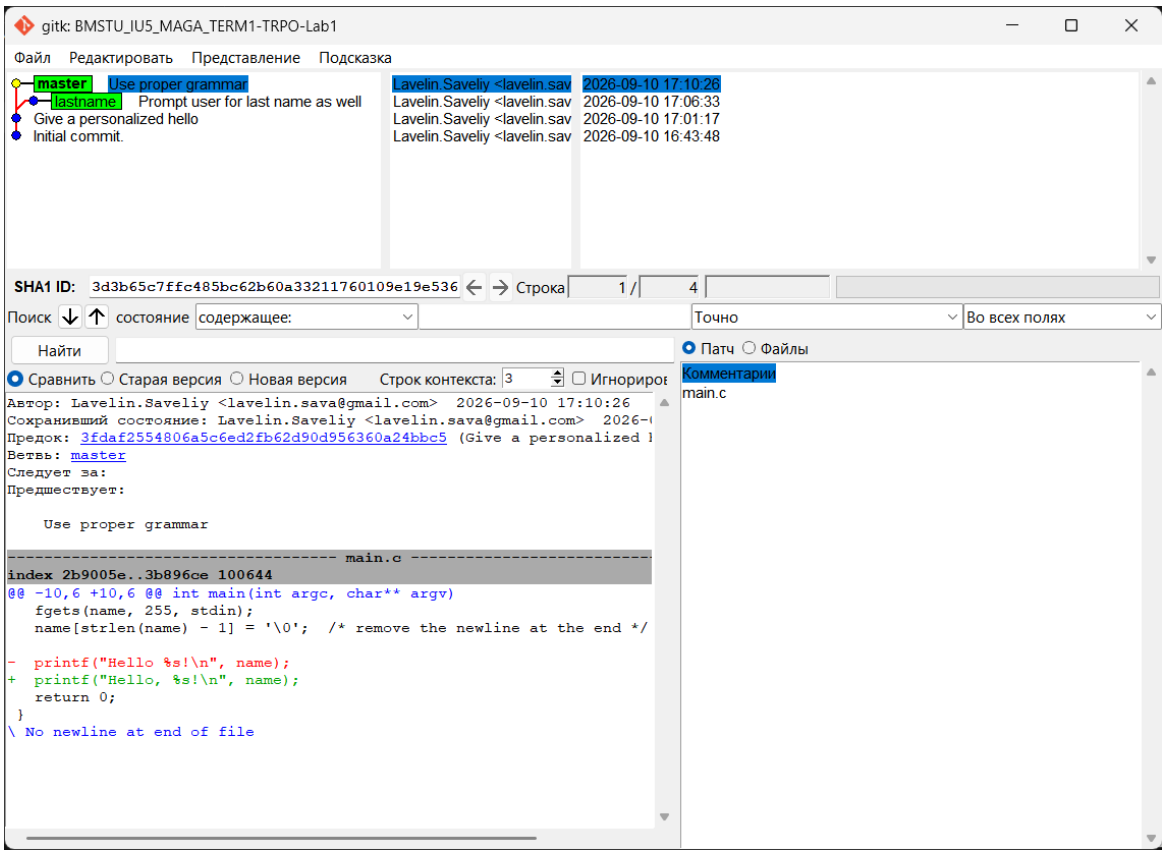


Рисунок 5 – Исправление в ветке master и история ветвления

запрос имени и фамилии (из ветки lastname) вместе с исправленной грамматикой (из ветки master). После разрешения конфликта изменения проиндексированы и зафиксирован коммит слияния «Merge branch ’lastname’». На рисунке 6 показана фиксация слияния веток.

2.5 Просмотр истории

Для демонстрации возможностей просмотра истории код запроса имени и фамилии вынесен из main.c в отдельную функцию askname() и отдельные файлы askname.c и askname.h. Изменение зафиксировано коммитом «Move the ask name code into its own function and file.». На рисунке 7 показана данная фиксация в gitk, а также использование истории для поиска изменений по нескольким файлам.

Просмотр истории отдельного файла (в том числе через Repository => Browse master’s Files и git blame) позволяет установить автора и коммит, в котором была добавлена или перемещена конкретная строка кода. Это особенно полезно при совместной разработке несколькими участниками.

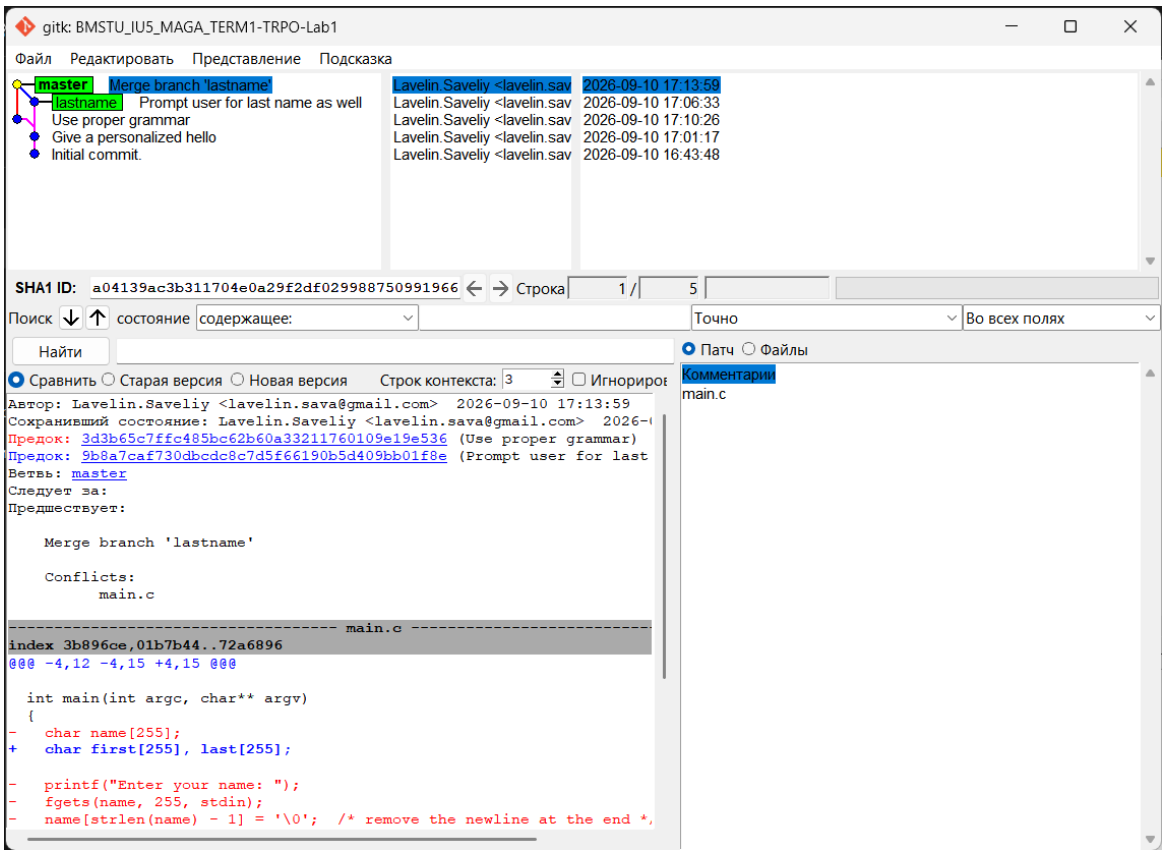


Рисунок 6 – Фиксация слияния ветки lastname в master с разрешением конфликта

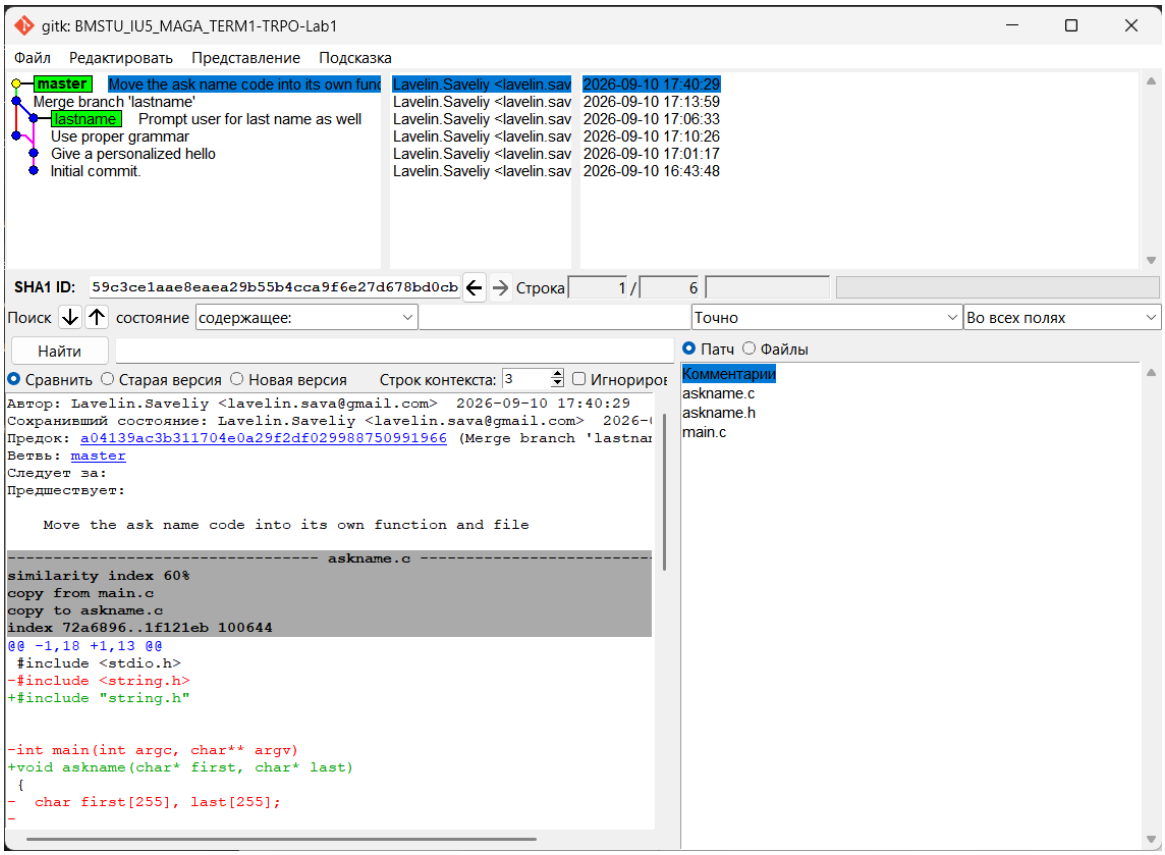


Рисунок 7 – Фиксация выноса кода в отдельные файлы askname.c и askname.h

2.6 Отмена изменений (revert)

Продемонстрирован откат изменений через создание обратного коммита (revert): для коммита «Move the ask name code into its own function and file.» выполнена команда «Revert this commit». При этом создаётся новая фиксация, отменяющая внесённые изменения, а сама история репозитория не переписывается. На рисунке 8 показан результат отката изменений.

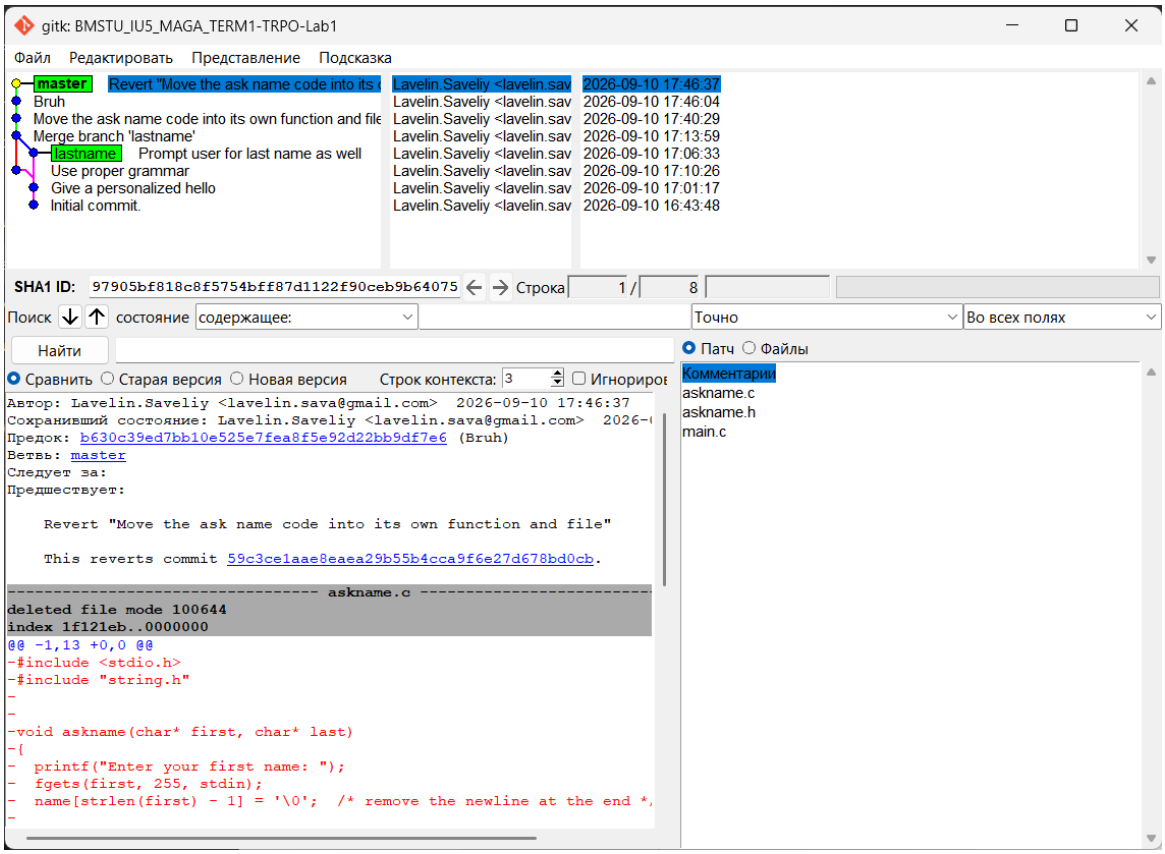


Рисунок 8 – Фиксация отката изменений (revert)

2.7 Публикация изменений на удалённом репозитории (GitHub)

Для совместной разработки локальный репозиторий опубликован на удалённом сервере GitHub: на сайте создан новый удалённый репозиторий, после чего с помощью команды push в него отправлены ветки локального репозитория. На рисунке 9 показано содержимое опубликованного репозитория на GitHub: список файлов проекта (main.c, askname.c, askname.h) с сообщениями и датами последних фиксаций для каждого файла.

Далее продемонстрирована коллективная разработка через механизм форков и pull request’ов. Члены команды сделали форк репозитория и внесли в свою

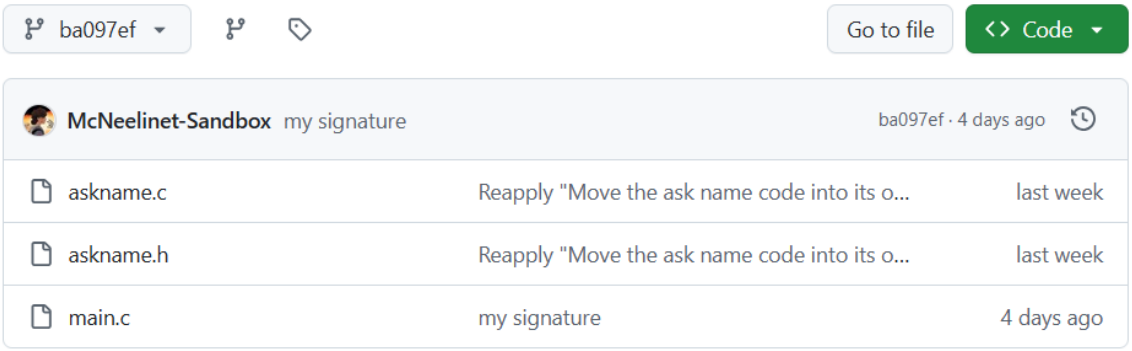


Рисунок 9 – Содержимое репозитория, опубликованного на GitHub

копию изменения. Был оформлен pull request на слияние. На рисунке 10 приведён список открытых pull request'ов репозитория на GitHub.

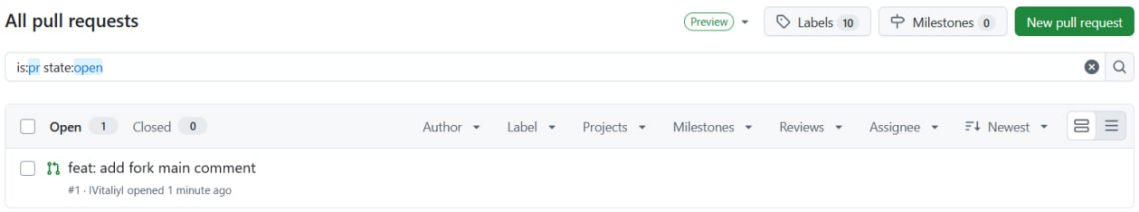


Рисунок 10 – Список открытых pull request'ов репозитория на GitHub

После проверки предложенных изменений pull request был принят и слит в ветку master. На рисунке 11 в окне gitk показана итоговая история удалённой ветки: слияние отображено отдельной фиксацией «Merge pull request #1 from IVitaliyl/master», а в нижней части окна приведён diff, демонстрирующий добавленную участником строку в файле main.c.

Таким образом, продемонстрирован полный цикл коллективной разработки через GitHub: публикация локального репозитория на удалённом сервере, внесение изменений сторонним участником через форк и pull request, проверка и слияние предложенных изменений владельцем репозитория, а также просмотр итоговой истории слияний и различий, внесённых разными участниками проекта.

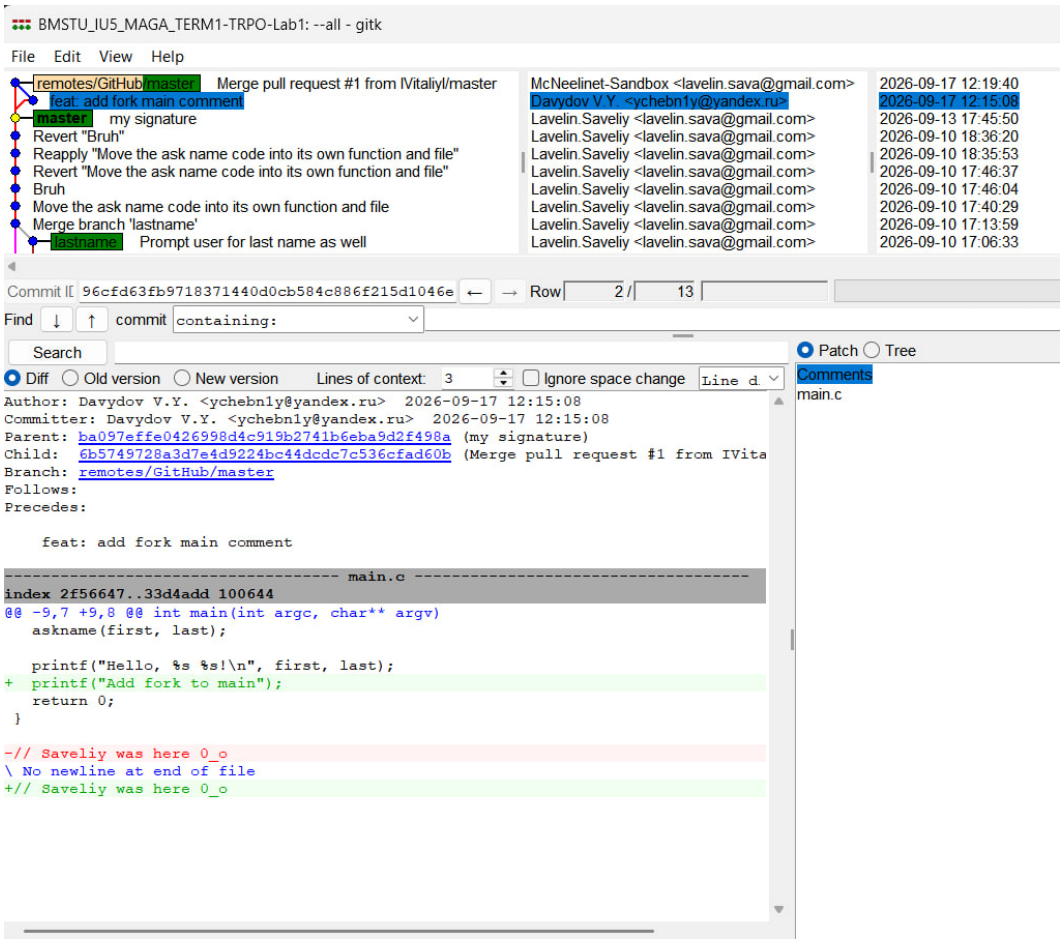


Рисунок 11 – История удалённой ветки после слияния pull request’a (gitk)

3 Заключение

В ходе выполнения лабораторной работы были изучены основные принципы организации разработки программного обеспечения с использованием методологии Scrum и получены практические навыки управления проектом в Jira.

В рамках работы был создан Scrum-проект, сформирован состав команды и распределены роли участников. Была создана и приоритизирована очередь продукта, выполнена оценка пользовательских историй, сформирован выпуск и определён состав первого спринта.

В процессе планирования спринта пользовательские истории были декомпозированы на отдельные задачи, для которых были заданы оценки трудоёмкости и назначены исполнители. В ходе моделирования выполнения спринта изменялись состояния задач, фиксировался прогресс работы и отслеживался оставшийся объём работ.

Для контроля состояния проекта были использованы запросы, отчёты и сводная панель Jira. Также были рассмотрены процедуры проведения Sprint Review и ретроспективы, завершения спринта и переноса незавершённых задач в следующую итерацию.

Таким образом, в результате лабораторной работы были освоены основные этапы управления программным проектом по Scrum: от формирования Product Backlog и планирования выпуска до организации спринта, контроля выполнения работ и подготовки следующей итерации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Словарь терминов для Git и GitHub // HTML Academy: [сайт]. – URL: https://htmlacademy.ru/blog/articles/git-and-github-glossary (дата обращения: 17.09.2026).

2. Git Wizardry // Habr: [сайт]. – URL: http://habrahabr.ru/post/60347/ (дата обращения: 17.09.2026).

3. Инструментальные средства управления версиями // КиберПедия: [сайт]. – URL: https://cyberpedia.su/11x9efe.html (дата обращения: 17.09.2026).

4. Git: наглядная справка // Mark Lodato's blog: [сайт]. – URL: http://marklodato.github.io/visual-git-guide/index-ru.html (дата обращения: 17.09.2026).

5. Основы работы с Git // Calculate Linux: [сайт]. – URL: http://www.calculate-linux.ru/main/ru/git (дата обращения: 17.09.2026).

6. Git Workflow // Habr: [сайт]. – URL: http://habrahabr.ru/post/60030/ (дата обращения: 17.09.2026).

7. Gitflow Workflow // Atlassian Git Tutorial: [сайт]. – URL: https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow (дата обращения: 17.09.2026).

8. Виноградова М.В., Авдеев Ю.В., Ваняшкин Ю.Ю., Мурашко И.А., Оганесян Р.Р., Пинская Н.М. Методические указания к лабораторным работам по курсу «Технологии разработки программного обеспечения». Лабораторная работа №2 «Управление версиями и коллективной разработкой программного проекта на примере утилиты Git». – М.: МГТУ им. Н.Э. Баумана, 2022.